Name: Aryan Sharma
Reg. No.: 23BCE1320
Subject: Cryptography & Network Security Lab
Assignment: Lab 6

Codes:
RSAUtils.java

```
public class RSAUtils {

    public static int gcd(int a, int b) {
        System.out.println("\nComputing GCD(" + a + ", " + b + ")");
        while (b != 0) {
            int temp = b;
            b = a % b;
            a = temp;
            System.out.println("a = " + a + ", b = " + b);
        }
        System.out.println("Final GCD = " + a);
        return a;
    }

    public static int[] extended_euclidean(int a, int b) {
        if (b == 0)
            return new int[]{a, 1, 0};

        int[] vals = extended_euclidean(b, a % b);
        int d = vals[0];
        int x = vals[2];
        int y = vals[1] - (a / b) * vals[2];

        return new int[]{d, x, y};
    }

    public static int mod_inverse(int e, int phi) {
        System.out.println("\nComputing Modular Inverse of " + e + " mod " + phi);
        int[] vals = extended_euclidean(e, phi);
        int gcd = vals[0];
        int x = vals[1];

        if (gcd != 1)
            throw new ArithmeticException("Inverse does not exist.");

        int result = (x % phi + phi) % phi;
        System.out.println("Private exponent d = " + result);
        return result;
    }

    public static long mod_exp(long base, long exponent, long modulus) {
        System.out.println("\nPerforming Modular Exponentiation:");
        System.out.println("Base = " + base + ", Exponent = " + exponent + ", Modulus
= " + modulus);

        long result = 1;
        base = base % modulus;

        while (exponent > 0) {
            if ((exponent & 1) == 1) {
                result = (result * base) % modulus;
                System.out.println("Result updated to " + result);
            }
            base = (base * base) % modulus;
```

```
            exponent >>= 1;
        }
        return result;
    }

    public static long encrypt(long message, long e, long n) {
        System.out.println("\nEncrypting Message: " + message);
        long cipher = mod_exp(message, e, n);
        System.out.println("Ciphertext = " + cipher);
        return cipher;
    }

    public static long decrypt(long cipher, long d, long n) {
        System.out.println("\nDecrypting Cipher: " + cipher);
        long message = mod_exp(cipher, d, n);
        System.out.println("Decrypted Message = " + message);
        return message;
    }
}
```

RSAServer.java

```
import java.io.*;
import java.net.*;

public class RSAServer {

    public static void main(String[] args) throws Exception {

        int p = 11;
        int q = 13;

        int n = p * q;
        int phi = (p - 1) * (q - 1);

        int e = 7;

        System.out.println("========== RSA KEY GENERATION ==========");
        System.out.println("Chosen primes:");
        System.out.println("p = " + p);
        System.out.println("q = " + q);

        System.out.println("\nn = p * q = " + n);
        System.out.println("phi(n) = (p-1)(q-1) = " + phi);

        RSAUtils.gcd(e, phi);
        int d = RSAUtils.mod_inverse(e, phi);

        System.out.println("\nPublic Key (e, n) = (" + e + ", " + n + ")");
        System.out.println("Private Key (d, n) = (" + d + ", " + n + ")");

        System.out.println("\n========== HOMOMORPHIC PROPERTY TEST ==========");

        int m1 = 4;
        int m2 = 5;

        long c1 = RSAUtils.encrypt(m1, e, n);
        long c2 = RSAUtils.encrypt(m2, e, n);

        long c_mul = (c1 * c2) % n;
        System.out.println("\nCipher Multiplication mod n = " + c_mul);

        long decryptedMul = RSAUtils.decrypt(c_mul, d, n);
```

```
        long expected = (m1 * m2) % n;

        System.out.println("\n(m1 * m2) mod n = " + expected);

        if (decryptedMul == expected)
            System.out.println("Homomorphic Property VERIFIED");
        else
            System.out.println("Homomorphic Property NOT VERIFIED");

        ServerSocket serverSocket = new ServerSocket(5000);
        System.out.println("\nServer waiting on port 5000...");

        Socket socket = serverSocket.accept();
        System.out.println("Client connected.");

        BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));

        long receivedCipher = Long.parseLong(in.readLine());

        System.out.println("\nEncrypted message received from client: " +
receivedCipher);

        long decryptedMessage = RSAUtils.decrypt(receivedCipher, d, n);

        System.out.println("\nFinal Decrypted Message at Server: " +
decryptedMessage);

        socket.close();
        serverSocket.close();
    }
}
```

RSAClient.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class RSAClient {

    public static void main(String[] args) throws Exception {

        int e = 7;
        int n = 143;

        Scanner sc = new Scanner(System.in);

        System.out.println("========== CLIENT SIDE ==========");
        System.out.print("Enter message (M < 143): ");
        long message = sc.nextLong();

        long cipher = RSAUtils.encrypt(message, e, n);

        Socket socket = new Socket("localhost", 5000);

        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        out.println(cipher);

        System.out.println("\nEncrypted message sent to server.");

        socket.close();
    }
}
```

Output:
ServerSide

```
Server waiting on port 5000...
Client connected.

Encrypted message received from client: 89

Decrypting Cipher: 89

Performing Modular Exponentiation:
Base = 89, Exponent = 103, Modulus = 143
Result updated to 89
Result updated to 122
Result updated to 67
Result updated to 122
Result updated to 67
Decrypted Message = 67

Final Decrypted Message at Server: 67
◆ lab6 ϟ main ⊙
> pwd

Path
----
C:\Users\aryan\Workspace\EDU\semester-6\cryptography\cryptography-network-security-lab\lab6
```

ClientSide

```
◆ lab6 ϟ main ⊙
> java RSAClient
========== CLIENT SIDE ==========
Enter message (M < 143): 67

Encrypting Message: 67

Performing Modular Exponentiation:
Base = 67, Exponent = 7, Modulus = 143
Result updated to 67
Result updated to 34
Result updated to 89
Ciphertext = 89

Encrypted message sent to server.
◆ lab6 ϟ main ⊙
> pwd

Path
----
C:\Users\aryan\Workspace\EDU\semester-6\cryptography\cryptography-network-security-lab\lab6
```